In [1]:
```
pip install -qU "langchain[google-genai]"
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [5]:
```
import getpass
import os

if not os.environ.get("GOOGLE_API_KEY"):
  os.environ["GOOGLE_API_KEY"] = "AIzaSyCOkBklj50fOOgbg8dASbUyn9Vb2jrGLuI"

from langchain.chat_models import init_chat_model

model = init_chat_model("gemini-2.5-flash", model_provider="google_genai")
```

In [3]:
```
model.invoke("today new in hyderabd")
```

Out[3]:
```
AIMessage(content='As an AI, I don\'t have real-time access to breaking news or minute-by-minute updates for a specific city like Hyderabad. My knowledge is based on information available up to my last training update.\n\nHowever, I can tell you the best ways to find out what\'s new today in Hyderabad:\n\n1.  **Local News Websites & Apps:**\n    *   **English:** The Times of India (Hyderabad edition), The Hindu (Hyderabad edition), Deccan Chronicle, Telangana Today.\n    *   **Telugu:** Sakshi, Eenadu, Namasthe Telangana.\n    *   Many of these also have apps for quick updates.\n\n2.  **Social Media:**\n    *   **Twitter:** Search for hashtags like #HyderabadNews, #Hyderabad, #HyderabadEvents. Follow local news outlets and prominent Hyderabad-based accounts.\n    *   **Facebook:** Join local Hyderabad community groups or follow pages of news channels and event organizers.\n\n3.  **Google Search:**\n    *   Simply search for "Hyderabad news today" or "What\'s new in Hyderabad today."\n    *   You can also get more specific: "new restaurants Hyderabad," "events in Hyderabad today," "traffic updates Hyderabad."\n\n4.  **Local Event Listing Websites/Apps:**\n    *   **BookMyShow, Paytm Insider:** For movies, concerts, plays, and other ticketed events.\n    *   **LBB Hyderabad (Little Black Book):** Often lists new cafes, stores, and unique events.\n\n**Common categories of "new" you might find:**\n\n*   **Political or Government Announcements:** Any new policies, projects, or statements from the state government.\n*   **Infrastructure Developments:** Updates on metro, flyovers, roads, or other major projects.\n*   **Business & Economy:** New store openings, corporate announcements, startup news.\n*   **Events & Entertainment:** Concerts, exhibitions, festivals, workshops, new movie releases.\n*   **Food & Drink:** New restaurant or cafe openings, special food festivals.\n*   **Weather Updates:** Important advisories or changes in weather patterns.\n*   **Traffic Alerts:** Major disruptions or diversions.\n\nTo get the most current information, I recommend checking one of the sources above right now!', additional_kwargs={}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.5-flash', 'safety_ratings': []}, id='run--0c3f0bce-eb07-4e6a-b68c-497d4d99ed81-0', usage_metadata={'input_tokens': 7, 'output_tokens': 1349, 'total_tokens': 1356, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 872}})
```

In [4]:
```
model.invoke("what the time in india")
```

Out[4]:
```
AIMessage(content="India observes Indian Standard Time (IST), which is UTC+5:30.\n\nAs of right now, if my current time is [Your current UTC time], then the time in India would be:\n\n**[Current UTC time] + 5 hours 30 minutes**\n\nFor example, if it's 10:00 AM UTC, it would be 3:30 PM in India.\n\nPlease tell me your current UTC time or your local time, and I can give you the exact time in India. Or, you can simply add 5 hours and 30 minutes to UTC to get the current time in India.", additional_kwargs={}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.5-flash', 'safety_ratings': []}, id='run--5f5b0ff9-438e-4258-93de-a7b9c30fe293-0', usage_metadata={'input_tokens': 6, 'output_tokens': 673, 'total_tokens': 679, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 538}})
```

In [5]:
```
model.invoke("what the time in india current")
```

Out[5]:
```
AIMessage(content='The current time in India is:\n\n**[Current Time]** on **[Current Date]** (India Standard Time, IST)\n\nIndia uses a single time zone, which is **UTC+5:30**.', additional_kwargs={}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.5-flash', 'safety_ratings': []}, id='run--918b8482-78fa-42e5-85a4-5c8f38e24868-0', usage_metadata={'input_tokens': 7, 'output_tokens': 840, 'total_tokens': 847, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 794}})
```

In [7]:
```
model.invoke("can you write machine learning alogorithm")
```

Out[7]:
```
AIMessage(content='That\'s a great request! When people ask for a "machine learning algorithm," they usually mean one of two things:\n\n1.  **A conceptual explanation of how an algorithm works.**\n2.  **A Python implementation using common libraries like `scikit-learn`**, which is the standard way to "write" ML algorithms in practice, as these libraries handle the complex mathematical optimizations under the hood.\n\nI\'ll provide examples of both, focusing on two fundamental algorithms: **Linear Regression** (for predicting continuous values) and **K-Nearest Neighbors (KNN)** (for classification).\n\n---\n\n## 1. Linear Regression (for Regression Problems)\n\n**Goal:** Predict a continuous target variable based on one or more input features. It assumes a linear relationship between inputs and the output.\n\n### Conceptual Explanation:\n\nImagine you have data points on a graph, where the x-axis is an input feature (e.g., "hours studied") and the y-axis is the output (e.g., "exam score"). Linear Regression tries to find the "best fit" straight line through these points.\n\n*   **The Model:** The equation of a straight line is typically written as $y = mx + b$, where:\n    *   `y` is the predicted output (exam score).\n    *   `x` is the input feature (hours studied).\n    *   `m` is the slope of the line (how much `y` changes for a one-unit change in `x`).\n    *   `b` is the y-intercept (the value of `y` when `x` is 0).\n    *   For multiple features, it becomes $y = b_0 + b_1x_1 + b_2x_2 + ... + b_nx_n$.\n\n*   **The Learning (Training) Process:** The algorithm\'s job is to find the optimal values for `m` (or $b_1, ..., b_n$) and `b` (or $b_0$). It does this by minimizing the difference between the predicted `y` values (from the line) and the actual `y` values in your training data. A common way to measure this difference is the **Mean Squared Error (MSE)**, which calculates the average of the squared differences between actual and predicted values. The algorithm uses techniques like **Gradient Descent** to iteratively adjust `m` and `b` to make MSE as small as possible.\n\n*   **Prediction:** Once `m` and `b` are learned, you can plug in a new `x` value (e.g., a student studied 7 hours) into the equation $y = mx + b$ to predict their `y` (exam score).\n\n### Python Implementation (using `scikit-learn`):\n\n```python\nimport numpy as np\nimport matplotlib.pyplot as plt\nfrom sklearn.linear_model import LinearRegression\nfrom sklearn.model_selection import train_test_split\nfrom sklearn.metrics import mean_squared_error, r2_score\n\n# 1. Generate some synthetic data for demonstration\nnp.random.seed(0)\nX = 2 * np.random.rand(100, 1) # 100 data points, 1 feature\ny = 4 + 3 * X + np.random.randn(100, 1) # y = 4 + 3x + noise\n\n# Visualize the generated data\nplt.figure(figsize=(8, 6))\nplt.scatter(X, y, color=\'blue\', label=\'Actual Data\')\nplt.title(\'Synthetic Data for Linear Regression\')\nplt.xlabel(\'Feature (X)\')\nplt.ylabel(\'Target (y)\')\nplt.legend()\nplt.grid(True)\nplt.show()\n\n# 2. Split the data into training and testing sets\n# This is crucial to evaluate how well the model generalizes to unseen data\nX_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)\n\nprint(f"Training data shape: X_train={X_train.shape}, y_train={y_train.shape}")\nprint(f"Testing data shape: X_test={X_test.shape}, y_test={y_test.shape}")\n\n# 3. Create a Linear Regression model instance\nmodel = LinearRegression()\n\n# 4. Train the model (fit the line to the training data)\nprint("\\nTraining the Linear Regression model...")\nmodel.fit(X_train, y_train)\nprint("Model training complete.")\n\n# 5. Make predictions on the test set\ny_pred = model.predict(X_test)\n\n# 6. Evaluate the model\'s performance\nmse = mean_squared_error(y_test, y_pred)\nr2 = r2_score(y_test, y_pred)\n\nprint(f"\\nModel Coefficients: {model.coef_[0][0]:.2f}") # This is \'m\'\nprint(f"Model Intercept: {model.intercept_[0]:.2f}") # This is \'b\'\nprint(f"Mean Squared Error (MSE) on test set: {mse:.2f}")\nprint(f"R-squared (R2) score on test set: {r2:.2f}")\n\n# 7. Visualize the regression line\nplt.figure(figsize=(8, 6))\nplt.scatter(X_test, y_test, color=\'blue\', label=\'Actual Test Data\')\nplt.plot(X_test, y_pred, color=\'red\', linewidth=3, label=\'Predicted Regression Line\')\nplt.title(\'Linear Regression: Actual vs. Predicted\')\nplt.xlabel(\'Feature (X)\')\nplt.ylabel(\'Target (y)\')\nplt.legend()\nplt.grid(True)\nplt.show()\n\n# Example of making a prediction on a new, unseen data point\nnew_X = np.array([[1.5]]) # Let\'s say a new data point has feature value 1.5\npredicted_y = model.predict(new_X)\nprint(f"\\nPrediction for new data point X={new_X[0][0]}: y={predicted_y[0][0]:.2f}")\n```\n\n---\n\n## 2. K-Nearest Neighbors (KNN) (for Classification Problems)\n\n**Goal:** Classify a new data point into one of several categories based on the classes of its `k` nearest neighbors in the training data.\n\n### Conceptual Explanation:\n\nImagine you have a bunch of fruit (apples and oranges) scattered on a table. You get a new, unknown fruit. How do you classify it?\n\n*   **The Model:** KNN is a "lazy learner" because it doesn\'t really "train" in the traditional sense of learning coefficients. Instead, it just stores all the training data.\n*   **The Learning (Training) Process:** Store all training data points along with their labels (e.g., "apple" or "orange").\n*   **Prediction:** When you want to classify a new, unknown data point:\n    1.  **Calculate Distance:** Measure the distance (e.g., Euclidean distance) from this new point to *every single point* in your training data.\n    2.  **Find K Nearest:** Identify the `k` training data points that are closest to your new point.\n    3.  **Vote:** Look at the labels of these `k` nearest neighbors. The new data point is assigned the class that is most common among its `k` neighbors (a majority vote).\n    *   **Example:** If `k=3` and the 3 nearest neighbors are 2 "apples" and 1 "orange", the new fruit is classified as an "apple."\n\n*   **Key Hyperparameter:** The choice of `k` (the number of neighbors) is crucial. A small `k` can be sensitive to noise, while a large `k` can blur class boundaries.\n\n### Python Implementation (using `scikit-learn`):\n\n```python\nimport numpy as np\nimport matplotlib.pyplot as plt\nfrom sklearn.neighbors import KNeighborsClassifier\nfrom sklearn.model_selection import train_test_split\nfrom sklearn.metrics import accuracy_score\nfrom sklearn.datasets import make_classification # To generate synthetic classification data\n\n# 1. Generate some synthetic classification data\nX, y = make_classification(n_samples=100, n_features=2, n_informative=2,\n                           n_redundant=0, n_clusters_per_class=1, random_state=42)\n\n# Visualize the generated data\nplt.figure(figsize=(8, 6))\nplt.scatter(X[:, 0], X[:, 1], c=y, cmap=\'viridis\', s=50, edgecolors=\'k\', label=\'Actual Data\')\nplt.title(\'Synthetic Data for K-Nearest Neighbors Classification\')\nplt.xlabel(\'Feature 1\')\nplt.ylabel(\'Feature 2\')\nplt.colorbar(label=\'Class\')\nplt.grid(True)\nplt.show()\n\n# 2. Split the data into training and testing sets\nX_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)\n\nprint(f"Training data shape: X_train={X_train.shape}, y_train={y_train.shape}")\nprint(f"Testing data shape: X_test={X_test.shape}, y_test={y_test.shape}")\n\n# 3. Create a K-Nearest Neighbors Classifier instance\n# We choose k=3 neighbors here. This is a hyperparameter you can tune.\nk = 3\nmodel = KNeighborsClassifier(n_neighbors=k)\n\n# 4. Train the model (for KNN, this simply means storing the training data)\nprint(f"\\nTraining the K-Nearest Neighbors (k={k}) model...")\nmodel.fit(X_train, y_train)\nprint("Model training complete (data stored).")\n\n# 5. Make predictions on the test set\ny_pred = model.predict(X_test)\n\n# 6. Evaluate the model\'s performance\naccuracy = accuracy_score(y_test, y_pred)\nprint(f"\\nAccuracy on test set (k={k}): {accuracy:.2f}")\n\n# 7. Visualize the decision boundary (more complex, but illustrative)\ndef plot_decision_boundary(X, y, model, title):\n    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1\n    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1\n    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),\n                         np.arange(y_min, y_max, 0.02))\n    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])\n    Z = Z.reshape(xx.shape)\n    plt.figure(figsize=(8, 6))\n    plt.contourf(xx, yy, Z, alpha=0.8, cmap=\'viridis\')\n    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=\'viridis\', s=50, edgecolors=\'k\')\n    plt.title(title)\n    plt.xlabel(\'Feature 1\')\n    plt.ylabel(\'Feature 2\')\n    plt.show()\n\nplot_decision_boundary(X_test, y_test, model, f\'KNN (k={k}) Decision Boundary on Test Data\')\n\n# Example of making a prediction on a new, unseen data point\nnew_data_point = np.array([[0.5, 0.5]]) # A new point to classify\npredicted_class = model.predict(new_data_point)\nprint(f"\\nPrediction for new data point {new_data_point[0]}: Class={predicted_class[0]}")\n```\n\n---\n\n### Key Takeaways for "Writing" ML Algorithms:\n\n1.  **Data is King:** The quality and quantity of your data are paramount. ML algorithms learn from data.\n2.  **Preprocessing:** Real-world data almost always needs cleaning, transformation, and scaling before being fed to an algorithm.\n3.  **Train/Test Split:** Always split your data to evaluate your model\'s ability to generalize to unseen data.\n4.  **Model Selection:** There are many algorithms; the choice depends on your problem (regression, classification, clustering), data type, and performance requirements.\n5.  **Hyperparameters:** Most algorithms have parameters (like `k` in KNN) that are not learned from data but set *before* training. Tuning these is crucial for optimal performance.\n6.  **Evaluation Metrics:** Choose appropriate metrics (e.g., MSE, R2 for regression; Accuracy, Precision, Recall, F1-score for classification) to assess your model.\n7.  **Libraries like `scikit-learn`** abstract away the complex math, allowing you to focus on data preparation, model selection, and evaluation. Implementing these from scratch is a good academic exercise but rarely done in production.\n\nThis should give you a solid foundation for understanding and implementing basic machine learning algorithms!', additional_kwargs={}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.5-flash', 'safety_ratings': []}, id='run--2aa2069f-088c-4fd4-9cc4-fa59ab6debd1-0', usage_metadata={'input_tokens': 9, 'output_tokens': 4334, 'total_tokens': 4343, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 1367}})
```

In [8]:
```
model.invoke("can you write machine learning alogorithm svm")
```

Out[8]:
```
AIMessage(content='Okay, let\'s break down the Support Vector Machine (SVM) algorithm.\n\n## Support Vector Machine (SVM) Algorithm\n\nThe Support Vector Machine (SVM) is a powe
rful and versatile machine learning algorithm capable of performing **classification**, **regression**, and even **outlier detection**. It is primarily used for **classification
** tasks.\n\n**Core Idea:**\nThe fundamental idea behind SVM is to find the "best" possible hyperplane that separates data points belonging to different classes in the feature s
pace. The "best" hyperplane is defined as the one that has the largest margin between the two classes.\n\n---\n\n### 1. The Goal: Optimal Separating Hyperplane\n\nImagine you ha
ve two types of data points (e.g., apples and oranges) scattered on a 2D plane. Your goal is to draw a line that separates them. You could draw many lines. SVM aims to find the
*one* line that maximizes the distance to the nearest data point of *any* class.\n\n*   **Hyperplane:** In a 2-dimensional space, it\'s a line. In a 3-dimensional space, it\'s a
plane. In higher-dimensional spaces, it\'s called a hyperplane.\n*   **Margin:** The distance between the hyperplane and the nearest data point from each class. SVM tries to max
imize this margin.\n*   **Support Vectors:** The data points that lie closest to the hyperplane (i.e., on the margin). These points are crucial because they "support" or define
the position and orientation of the optimal hyperplane. If you remove these support vectors, the hyperplane might change.\n\n---\n\n### 2. How it Works (Linear SVM - Hard Margi
n)\n\nLet\'s start with the simplest case: linearly separable data.\n\n1.  **Identify Classes:** You have data points belonging to at least two classes (SVM is inherently a bina
ry classifier, but can be extended to multi-class).\n2.  **Define a Hyperplane:** A hyperplane in a `p`-dimensional space can be represented by the equation:\n    `w · x + b = 0
`\n    where:\n    *   `w` is the normal vector to the hyperplane (a vector perpendicular to the hyperplane).\n    *   `x` is a data point (feature vector).\n    *   `b` is the
bias term (intercept).\n3.  **Define Margin Boundaries:** For each class, there\'s a parallel hyperplane:\n    *   `w · x + b = +1` (for points of class 1)\n    *   `w · x + b =
-1` (for points of class -1)\n    The points lying on these planes are the support vectors.\n4.  **Maximize the Margin:** The distance between these two margin-defining hyperpla
nes is `2 / ||w||`. To maximize this distance, SVM minimizes `||w||` (or `1/2 ||w||^2` for mathematical convenience).\n5.  **Constraints:** This minimization is subject to the c
onstraint that all data points are correctly classified and lie on the correct side of their respective margin boundaries:\n    *   For `y_i = +1` (class 1): `w · x_i + b ≥ +1`
\n    *   For `y_i = -1` (class -1): `w · x_i + b ≤ -1`\n    These two can be combined into: `y_i (w · x_i + b) ≥ 1` for all data points `i`.\n6.  **Optimization Problem:** SVM s
olves a convex optimization problem (typically using quadratic programming) to find the `w` and `b` that satisfy these conditions and maximize the margin.\n\n---\n\n### 3. Exten
sions and Key Concepts\n\n#### a. Soft Margin SVM (Handling Non-Separable Data & Outliers)\n\nWhat if the data isn\'t perfectly linearly separable, or there are outliers that ma
ke a hard margin impossible/undesirable? This is where **Soft Margin SVM** comes in.\n\n*   **Slack Variables (ξ - epsilon):** SVM introduces "slack variables" (`ξ_i`) for each
data point. These variables allow some data points to be on the wrong side of the margin or even the wrong side of the hyperplane.\n    *   `ξ_i = 0`: Point is correctly classif
ied and outside the margin.\n    *   `0 < ξ_i < 1`: Point is correctly classified but *inside* the margin.\n    *   `ξ_i ≥ 1`: Point is misclassified.\n*   **Cost Parameter (C):
** A regularization parameter `C` is introduced. It controls the trade-off between maximizing the margin and minimizing the misclassification error.\n    *   **Small C:** Allows
a wider margin, even if it means more misclassifications (tolerates more errors, less prone to overfitting).\n    *   **Large C:** Penalizes misclassifications heavily, leading
to a narrower margin but fewer errors (can lead to overfitting).\n*   **Modified Optimization:** The objective now becomes: Minimize `1/2 ||w||^2 + C * Σ ξ_i` subject to the con
straints `y_i (w · x_i + b) ≥ 1 - ξ_i` and `ξ_i ≥ 0`.\n\n#### b. The Kernel Trick (Handling Non-Linear Data)\n\nMany real-world datasets are not linearly separable in their orig
inal feature space. The **Kernel Trick** is a brilliant solution to this.\n\n*   **Idea:** Instead of trying to find a linear boundary in the original low-dimensional space, SVM
implicitly maps the data into a higher-dimensional feature space where it *might* become linearly separable.\n*   **Kernel Function:** A kernel function `K(x_i, x_j)` calculates
the dot product of the transformed feature vectors in the higher-dimensional space *without explicitly performing the transformation*. This avoids the computational cost of work
ing in very high dimensions.\n*   **Common Kernel Functions:**\n    *   **Polynomial Kernel:** `K(x_i, x_j) = (γ * x_i · x_j + r)^d`\n    *   **Radial Basis Function (RBF) / Gau
ssian Kernel:** `K(x_i, x_j) = exp(-γ * ||x_i - x_j||^2)` (most common and powerful)\n    *   **Sigmoid Kernel:** `K(x_i, x_j) = tanh(γ * x_i · x_j + r)`\n*   **Hyperparameter `
γ` (gamma):** For RBF and Polynomial kernels, `γ` defines how much influence a single training example has.\n    *   **Small `γ`:** Large influence, smoother decision boundar
y.\n    *   **Large `γ`:** Small influence, can lead to complex, wiggly decision boundaries and overfitting.\n\n---\n\n### 4. Algorithm Summary (High-Level Steps)\n\n1.  **Inpu
t:** Training data `(x_i, y_i)` where `x_i` are feature vectors and `y_i` are class labels (+1 or -1).\n2.  **Choose Kernel:** Select an appropriate kernel function (e.g., Linea
r, RBF, Polynomial) based on the data characteristics.\n3.  **Choose Hyperparameters:** Set `C` (regularization) and `γ` (kernel parameter, if applicable). These are often tuned
using techniques like cross-validation.\n4.  **Formulate Optimization Problem:** Construct the optimization problem to minimize `1/2 ||w||^2 + C * Σ ξ_i` subject to `y_i (w · x_
i + b) ≥ 1 - ξ_i` and `ξ_i ≥ 0`. When using a kernel, the problem is formulated in terms of dot products, which are then replaced by kernel function evaluations.\n5.  **Solve Op
timization:** Use a quadratic programming solver to find the optimal `w` and `b` (or more accurately, the Lagrange multipliers from the dual problem, which implicitly define `w`
and `b`).\n6.  **Identify Support Vectors:** The data points corresponding to non-zero `ξ_i` or `α_i` (Lagrange multipliers) are the support vectors.\n7.  **Construct Decision F
unction:** The learned hyperplane (or decision boundary in the original space) is defined by `f(x) = sign(w · x + b)` for linear SVM, or `f(x) = sign(Σ α_i y_i K(x_i, x) + b)` f
or kernel SVM.\n8.  **Prediction:** For a new data point `x_new`, apply the decision function `f(x_new)` to predict its class.\n\n---\n\n### 5. Advantages of SVM\n\n*   **Effect
ive in High-Dimensional Spaces:** Works well even when the number of features is greater than the number of samples.\n*   **Memory Efficient:** Only a subset of the training dat
a (support vectors) is used in the decision function.\n*   **Versatile:** Different kernel functions make it adaptable to various data types and non-linear problems.\n*   **Robu
st to Outliers:** Soft margin allows for some misclassifications, making it less sensitive to noise compared to hard margin.\n*   **Strong Theoretical Foundation:** Based on the
principle of structural risk minimization, leading to good generalization capabilities.\n\n### 6. Disadvantages of SVM\n\n*   **Computationally Intensive:** Can be slow to train
on very large datasets, especially with non-linear kernels.\n*   **Sensitive to Hyperparameter Tuning:** Performance heavily depends on the choice of `C`, `γ`, and the kernel fu
nction.\n*   **Less Interpretability:** The "black box" nature, especially with complex kernels, makes it harder to understand *why* a particular classification was made compare
d to, say, decision trees.\n*   **Does Not Directly Provide Probability Estimates:** Unlike logistic regression, SVM outputs a decision boundary, not probabilities (though exten
sions exist to estimate them).\n\n---\n\n### 7. Implementation (Python with scikit-learn)\n\n```python\nimport numpy as np\nimport matplotlib.pyplot as plt\nfrom sklearn import
datasets\nfrom sklearn.svm import SVC # Support Vector Classifier\nfrom sklearn.model_selection import train_test_split\nfrom sklearn.preprocessing import StandardScaler\nfrom m
atplotlib.colors import ListedColormap\n\n# 1. Generate some synthetic data (or load a real dataset)\nX, y = datasets.make_classification(n_samples=100,\n
n_features=2,\n                           n_redundant=0,\n                           n_informative=2,\n                           random_state=6,\n
n_clusters_per_class=1)\n\n# Split data into training and testing sets\nX_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)\n\n# 2. Preproc
essing (Scaling is often important for SVM, especially with RBF kernel)\nscaler = StandardScaler()\nX_train_scaled = scaler.fit_transform(X_train)\nX_test_scaled = scaler.transf
orm(X_test)\n\n# 3. Create and Train the SVM Model\n# Try different kernels: \'linear\', \'poly\', \'rbf\'\n# Tune C and gamma (for \'rbf\' and \'poly\')\nsvm_model = SVC(kernel
=\'rbf\', C=1.0, gamma=\'scale\', random_state=42)\n# \'gamma=\'scale\'\' uses 1 / (n_features * X.var()) as gamma\n# \'gamma=\'auto\'\' uses 1 / n_features\n\nsvm_model.fit(X_t
rain_scaled, y_train)\n\n# 4. Evaluate the Model\ntrain_accuracy = svm_model.score(X_train_scaled, y_train)\ntest_accuracy = svm_model.score(X_test_scaled, y_test)\n\nprint(f"Tr
aining Accuracy: {train_accuracy:.4f}")\nprint(f"Test Accuracy: {test_accuracy:.4f}")\n\n# 5. Visualize the Decision Boundary (for 2D data)\ndef plot_decision_boundary(X, y, cla
ssifier, scaler=None, title=\'\'):\n    h = .02  # step size in the mesh\n    cmap_light = ListedColormap([\'#FFAAAA\', \'#AAFFAA\', \'#AAAAFF\'])\n    cmap_bold = ListedColorma
p([\'#FF0000\', \'#00FF00\', \'#0000FF\'])\n\n    # Scale the data for plotting if a scaler was used during training\n    if scaler:\n        X_plot = scaler.transform(X)\n    e
lse:\n        X_plot = X\n\n    x_min, x_max = X_plot[:, 0].min() - 1, X_plot[:, 0].max() + 1\n    y_min, y_max = X_plot[:, 1].min() - 1, X_plot[:, 1].max() + 1\n    xx, yy = n
p.meshgrid(np.arange(x_min, x_max, h),\n                         np.arange(y_min, y_max, h))\n\n    Z = classifier.predict(np.c_[xx.ravel(), yy.ravel()])\n    Z = Z.reshape(xx.s
hape)\n\n    plt.figure(figsize=(8, 6))\n    plt.contourf(xx, yy, Z, cmap=cmap_light, alpha=0.8)\n\n    # Plot the training points\n    plt.scatter(X_plot[:, 0], X_plot[:, 1], c
=y, cmap=cmap_bold, edgecolor=\'k\', s=20)\n    plt.xlim(xx.min(), xx.max())\n    plt.ylim(yy.min(), yy.max())\n    plt.title(title)\n    plt.xlabel("Feature 1 (scaled)")\n    p
lt.ylabel("Feature 2 (scaled)")\n    plt.show()\n\nplot_decision_boundary(X, y, svm_model, scaler, title=f\'SVM Decision Boundary (Kernel={svm_model.kernel}, C={svm_model.C}, Ga
mma={svm_model.gamma})\')\n```\n\nThis comprehensive overview covers the core principles, mathematical intuition, key extensions, and practical considerations of the SVM algorit
hm.', additional_kwargs={}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.5-flash', 'safety_r
atings': []}, id='run--3590dfaf-670d-4975-a973-94d6b59b72de-0', usage_metadata={'input_tokens': 10, 'output_tokens': 4472, 'total_tokens': 4482, 'input_token_details': {'cache_r
ead': 0}, 'output_token_details': {'reasoning': 1325}})
```

In [9]:
```
model.invoke("what is love")
```

Out[9]:
```
AIMessage(content="Love is one of the most profound, complex, and universal human experiences. There isn't a single, definitive answer because it manifests in countless ways and
is deeply subjective, but here's an attempt to capture its essence:\n\nAt its core, **love is a deep and intense feeling of affection, care, and concern for another person, anim
al, idea, or even oneself.** It often involves a strong sense of connection, attachment, and a desire for the well-being and happiness of the object of one's love.\n\nHowever, l
ove is more than just a feeling; **it's also a set of behaviors, actions, and a commitment.** It's a dynamic force that can transform individuals and relationships.\n\nHere are
some key aspects and manifestations of love:\n\n1.  **Deep Affection and Connection:** It's a powerful emotional bond that draws you towards another, often accompanied by warmt
h, tenderness, and a sense of belonging.\n2.  **Care and Concern:** A fundamental aspect is genuinely wanting the best for the other, feeling their joys and sorrows, and being m
otivated to support and protect them.\n3.  **Empathy and Understanding:** Love often involves trying to see the world from the other's perspective, understanding their needs, fe
ars, and desires, and offering compassion.\n4.  **Acceptance and Forgiveness:** Loving someone means accepting them for who they are, flaws and all, and being willing to forgive
mistakes and move forward.\n5.  **Sacrifice and Selflessness:** Sometimes, love requires putting the needs or happiness of the loved one before your own, making compromises, or
enduring difficulties for their sake.\n6.  **Commitment and Trust:** Especially in romantic and familial love, it involves a pledge of loyalty, reliability, and faith in the oth
er person.\n7.  **Joy and Euphoria:** Being in love, or experiencing love, can bring immense happiness, excitement, and a sense of purpose.\n8.  **Vulnerability:** True love oft
en requires opening yourself up, being authentic, and allowing yourself to be seen, even the parts you might hide from others.\n\n**Different Forms of Love:**\n\n*   **Romantic
Love:** Characterized by passion, intimacy, and commitment. It often involves physical attraction, a deep emotional bond, and a desire for a shared future.\n*   **Familial Love:
** The strong, often unconditional, bond between family members (parents, children, siblings). It's built on shared history, protection, and deep belonging.\n*   **Platonic Lov
e:** The deep affection, respect, and connection between friends. It's based on shared interests, mutual support, and trust, without romantic or sexual elements.\n*   **Self-Lov
e:** The acceptance, respect, and care for oneself. It's crucial for mental health and involves setting boundaries, prioritizing one's well-being, and having a healthy self-este
em.\n*   **Universal Love (Agape):** A broader sense of compassion, empathy, and goodwill towards humanity or all living things. It's a desire for the well-being of others, even
strangers.\n\nUltimately, love is a dynamic, evolving force that brings meaning, joy, and often challenges to our lives. It's something we continuously learn, practice, and expe
rience throughout our lives, and its definition is as unique as each individual who feels it.", additional_kwargs={}, response_metadata={'prompt_feedback': {'block_reason': 0,
'safety_ratings': []}, 'finish_reason': 'STOP', 'model_name': 'gemini-2.5-flash', 'safety_ratings': []}, id='run--d48728f2-693d-4d43-9d9d-b02ce27a3348-0', usage_metadata={'input
_tokens': 4, 'output_tokens': 2015, 'total_tokens': 2019, 'input_token_details': {'cache_read': 0}, 'output_token_details': {'reasoning': 1309}})
```

In [6]:
```
from langchain_core.messages import HumanMessage, SystemMessage

messages = [
    SystemMessage("translate the following from english in telugu"),
    HumanMessage("how are you")
]
model.invoke(messages)
```

Out[6]:
```
AIMessage(content='మీరు ఎలా ఉన్నారు?', additional_kwargs={}, response_metadata={'prompt_feedback': {'block_reason': 0, 'safety_ratings': []}, 'finish_reason': 'STOP', 'model_na
me': 'gemini-2.5-flash', 'safety_ratings': []}, id='run--14de0e12-65a2-413a-90b1-7f4254da2cc0-0', usage_metadata={'input_tokens': 12, 'output_tokens': 5, 'total_tokens': 17, 'in
put_token_details': {'cache_read': 0}})
```

In [ ]:
```

```